CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 4

JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Student Name	Shreyas
Roll Number	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To make the StudentHub home page interactive with JavaScript (ES6+), using DOM manipulation and event handling, without breaking accessibility.

- Build six interactive components: a collapsible FAQ, a modal popup, an image/content slider, a dismissible notification banner, a responsive hamburger menu and a light/dark theme switcher.
- Select and change page elements with the DOM API (getElementById, querySelectorAll, classList, attributes, createElement) and react to user actions with addEventListener.
- Keep every component usable with the keyboard: real button elements, ARIA states, Escape to close, focus kept inside the modal.
- Add transition effects to cards and buttons (intermediate extension).
- Store the user's theme and banner preferences in localStorage and restore them when the page loads (advanced extension).
- Test every component in a browser, check the console for errors and record genuine screenshots.

2. Problem Definition

Official problem definition: *"Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript."*

The Practical 4 folder started as a copy of the StudentHub home page that opened as a blank page: it still contained a rule that hid the page until a login script ran, and that script was missing. It also used inline styles, an onclick attribute and images and fonts loaded from the internet. This practical repairs the page and rebuilds the six components so that they work offline, are written in modern JavaScript and can be operated with a keyboard.

The practical list asks four key questions; the answers for this implementation are:

Key question	Answer in this practical
How are DOM elements selected and modified?	Elements are selected with getElementById and querySelectorAll, and changed through the hidden property, classList.toggle, setAttribute (ARIA states), textContent, style.transform and createElement (Section 5).
Are event listeners attached correctly?	Yes. All listeners are attached in assets/js/script.js with addEventListener after DOMContentLoaded; the page has no onclick or other inline handlers (static check, Section 9.1).
Is localStorage used for remembering theme choice?	Yes. The theme is stored under the key sh_theme and restored in the page head before the page is drawn; the banner dismissal is stored under sh_banner_dismissed (Section 6).
Does interactivity improve usability without breaking accessibility?	The components use real buttons with ARIA states, Escape closes the menu and the modal, focus is managed, the slider can be paused, and reduced-motion settings are respected. Colour contrast passes in both themes (Section 9).

3. Tools and Technologies Used

Tool / technology	Use in this practical
HTML5	Page structure of index.html with semantic elements and ARIA attributes
CSS3	assets/css/style.css: colour variables for both themes, mobile-first media queries, transitions and a fade-in animation
JavaScript (ES6+)	assets/js/theme.js and assets/js/script.js: const/let, arrow functions, template literals, spread syntax
Web Storage API (localStorage)	Remembers the theme and the dismissed banner
Microsoft Edge 156.0.4314.50	Running and testing the page; headless mode used for the scripted tests and the screenshots
Python 3.12.0	source/check_page.py (static checks) and source/contrast_check.py (WCAG colour contrast), standard library only
Visual Studio Code, Git	Source code editor; version control of the StudentHub repository
Operating system	Windows 11

4. Implementation of the Six Components

All six components are on one page, Practical-4/index.html. The page has no inline CSS or JavaScript: styles are in assets/css/style.css, assets/js/theme.js is loaded in the head, and assets/js/script.js is loaded at the end of the body. The page works offline and uses only local SVG images and system fonts.

4.1 Collapsible FAQ

- **Markup:** three questions in the Quick FAQ section. Each question is a button inside an h3 with aria-expanded="false" and aria-controls pointing to its answer, which starts with the hidden attribute.
- **JavaScript:** querySelectorAll('.faq-question') selects the buttons and a click listener is added to each. A click first closes all answers, then opens the clicked one if it was closed, so only one answer is open at a time (accordion); clicking an open question closes it.
- **Transition:** the “+” icon after each question rotates 45° into a “×” when aria-expanded is true (CSS transition, 0.3 s).
- **Evidence:** Figure 2.

4.2 Modal popup

- **Markup:** an overlay div (hidden) contains an element with role="dialog", aria-modal="true" and aria-labelledby pointing to its heading, a close button (×) and an “I understand” button.
- **Opening:** the View portal guidelines button calls openModal(), which remembers the focused element, removes hidden and moves focus to the close button.
- **Closing:** ×, “I understand”, a click on the dark overlay (e.target === modal) or the Escape key call closeModal(), which hides the dialog and returns focus to the button that opened it.
- **Focus trap:** while the dialog is open, Tab on the last button moves to the first and Shift+Tab on the first moves to the last. The overlay fades in with a CSS animation.
- **Evidence:** Figure 3.

4.3 Image/content slider

- **Markup:** three slides (local SVG image and caption) with role="group" and labels “1 of 3” to “3 of 3”, plus Previous (‹), Next (›) and Pause/Play buttons.
- **Dots:** one dot button per slide is created with document.createElement and appendChild; the current dot gets aria-current="true".
- **JavaScript:** showSlide(index) moves every slide with style.transform = translateX(…%) and sets aria-hidden on the slides that are not shown. The index wraps around with the remainder operator, so Next on slide 3 shows slide 1.
- **Auto-play:** setInterval advances the slider every 5 seconds. Using Previous, Next or a dot stops it, and the Pause/Play button switches it off and on. While auto-playing the slider is not announced (aria-live="off"); after the user takes control it is announced politely.
- **Reduced motion:** if the system asks for reduced motion, the slider starts paused and slides change without the sliding transition.
- **Evidence:** Figure 4.

4.4 Dismissible notification banner

- **Markup:** a banner at the top of the page (role="region", aria-label="Announcement") with a × button labelled “Dismiss announcement”.
- **JavaScript:** the click listener sets banner.hidden = true, saves sh_banner_dismissed = 1 in localStorage and moves focus to the main content, so keyboard focus is not lost. When the page loads, the banner is hidden immediately if the key is set.
- **Evidence:** Figure 5 (page reloaded in a new browser session after dismissing the banner).

4.5 Responsive hamburger menu

- **Markup:** a ☰ Menu button with aria-expanded and aria-controls="nav-links", and the navigation list ul#nav-links with links to the sections of the page.
- **CSS:** below 768px the link list is hidden unless it has the class is-open; from 768px the Menu button is hidden and the links are shown in a row in the header.
- **JavaScript:** setMenu(open) calls classList.toggle('is-open', open) and updates aria-expanded. The menu closes when a link is chosen (one listener on the list checks e.target.closest('a')) and when Escape is pressed, which also returns focus to the Menu button.
- **Evidence:** Figure 7, at a width of 375px.

4.6 Light/dark theme switcher

- **CSS:** all colours are CSS custom properties. :root holds the light values and [data-theme="dark"] overrides them, so switching the theme only changes one attribute on the html element. Background and text colours change with a 0.3 s transition.
- **JavaScript:** theme.js, loaded in the head, reads sh_theme and sets data-theme before the page is drawn, so a saved dark theme does not flash white first. The Dark mode button calls window.toggleTheme(), which switches and saves the theme; script.js then changes the button text (Dark mode / Light mode) and aria-pressed.
- **Evidence:** Figure 6 (page reloaded in a new browser session after switching to dark).

4.7 Transition effects and other additions

- **Cards:** the three Portal sections cards lift by 6px with a stronger shadow on hover (transform and box-shadow transitions, 0.3 s).
- **Buttons:** buttons change colour on hover; the main buttons also lift slightly and the slider arrows grow (0.2 s transitions).
- **Reduced motion:** with prefers-reduced-motion: reduce, sliding, lifting and the modal fade are switched off; colour transitions remain.
- **Reset saved preferences:** a footer button removes both localStorage keys and reloads the page, so the persistence can be demonstrated again.
- **Skip link:** “Skip to main content” is the first link on the page and targets main#main-content.

5. JavaScript DOM Manipulation and Event Handling

assets/js/script.js wraps all code in one DOMContentLoaded listener, so the elements exist before they are selected. Each component checks that its elements were found (for example if (banner && closeBannerBtn)) before adding listeners, so a missing element cannot cause an error. The techniques used are:

Technique	Used for	Example in script.js
Selecting elements	Finding buttons, the banner, the modal, slides and the menu	getElementById('custom-modal'), querySelectorAll('.faq-question'), modal.querySelectorAll('button')
Showing and hiding	Banner, modal and FAQ answers	banner.hidden = true; answer.hidden = false
Class manipulation	Opening and closing the mobile menu	navLinks.classList.toggle('is-open', open)
Attributes (ARIA state)	Telling assistive technology what changed	setAttribute('aria-expanded', …), aria-pressed, aria-current, aria-hidden, aria-live
Text and style	Button labels and slide position	themeToggleBtn.textContent = 'Light mode'; slide.style.transform = `translateX(…)`
Creating elements	Slider dot buttons	document.createElement('button'), dotsBox.appendChild(dot)
Click events	All buttons	addEventListener('click', openModal)
Keyboard events	Escape for the menu and modal; Tab focus trap	document.addEventListener('keydown', …) with e.key, e.shiftKey, e.preventDefault()
Event delegation and targets	One listener for all menu links; overlay click	e.target.closest('a'); e.target === modal
Focus management	Modal, banner and menu	closeModalBtn.focus(); lastFocused.focus()
Timers and media queries	Slider auto-play; reduced motion	setInterval / clearInterval; matchMedia('(prefers-reduced-motion: reduce)')

ES6+ features used: const and let (no var), arrow functions, template literals, the spread operator ([...slides]), and Array methods such as forEach and map. The static check in Section 9.1 confirms that the page has no onclick or other inline event handlers.

6. localStorage for Theme and Banner Preferences

Key	Value	Written when	Read when
sh_theme	'light' or 'dark'	The theme button is clicked (window.toggleTheme in theme.js)	theme.js runs in the head, before the page is drawn
sh_banner_dismissed	'1'	The banner's × button is clicked	script.js runs on DOMContentLoaded

- **Safe access:** theme.js defines readPref and writePref, which wrap localStorage.getItem and setItem in try/catch. If the browser blocks storage, the page still works; the preference is just not remembered.
- **No flash of the wrong theme:** because theme.js is loaded in the head, the saved theme is applied before the body is rendered.
- **Reset:** the Reset saved preferences button calls localStorage.removeItem for both keys and reloads the page.
- **How to demonstrate:** click Dark mode and close the banner, then reload the page (F5) or open it again: the dark theme is restored and the banner stays hidden. Click Reset saved preferences to start again.

Only these two small preference values are stored. No personal data, passwords or form contents are written to localStorage.

7. Browser Screenshots

The screenshots below are genuine renders of Practical-4/index.html by Microsoft Edge 156 in headless mode. The state shown (an open FAQ answer, the open modal, and so on) was produced by the same scripted clicks that were used in the tests. The image files are in Practical-4/screenshots/.

The computer used reports the Windows setting "reduce motion" (animation effects off), so the slider starts paused and its button shows Play in these images. The slider's auto-play was tested separately with normal motion forced (Section 9.3).

7.1 Complete page – light theme, 1280px

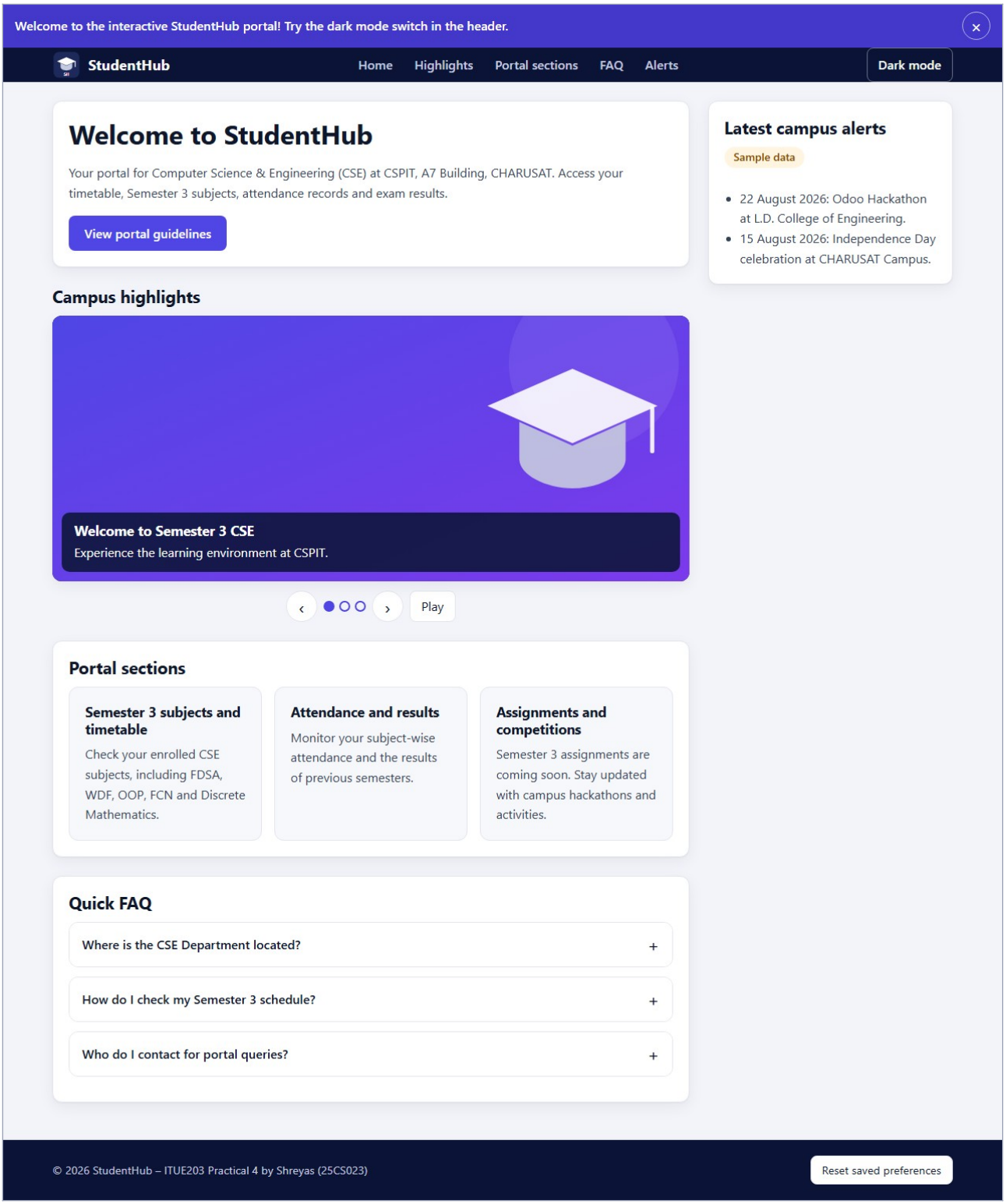


Figure 1: Complete page in the light theme at 1280px, first visit (banner shown) – page-desktop.png

7.2 Collapsible FAQ

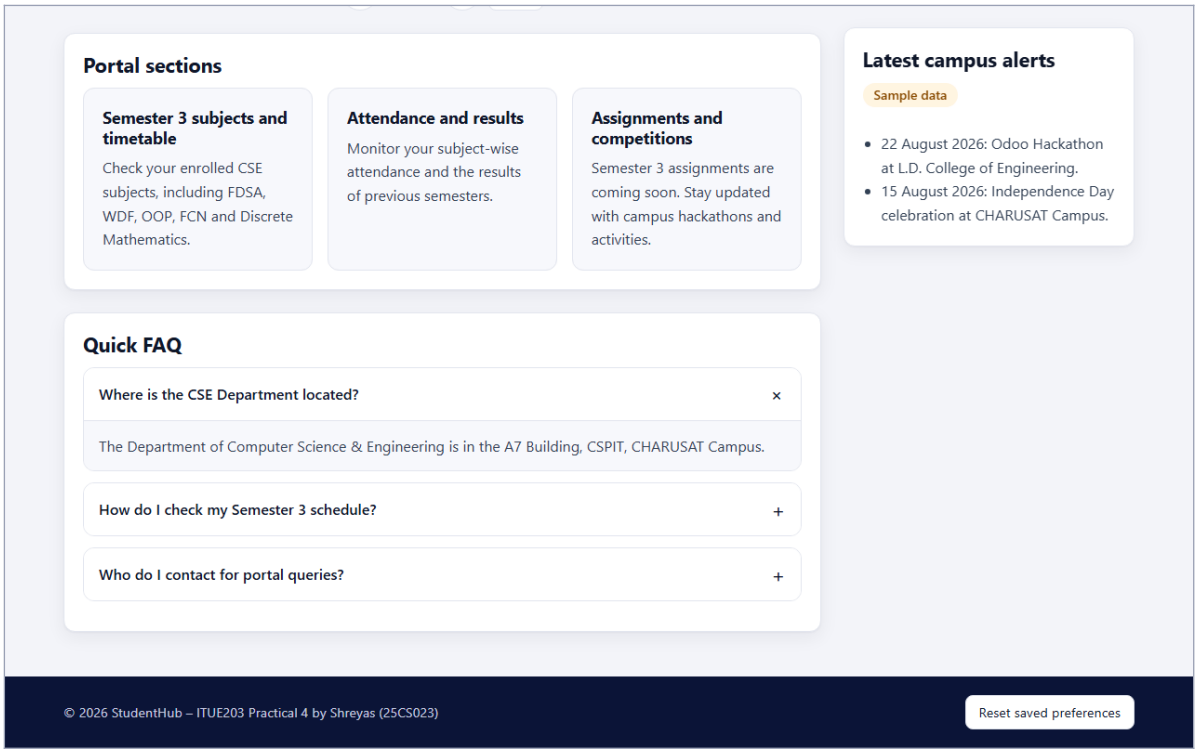


Figure 2: First FAQ question opened – the answer is shown and its "+" has turned into "×"; the other answers stay closed – faq.png

7.3 Modal popup

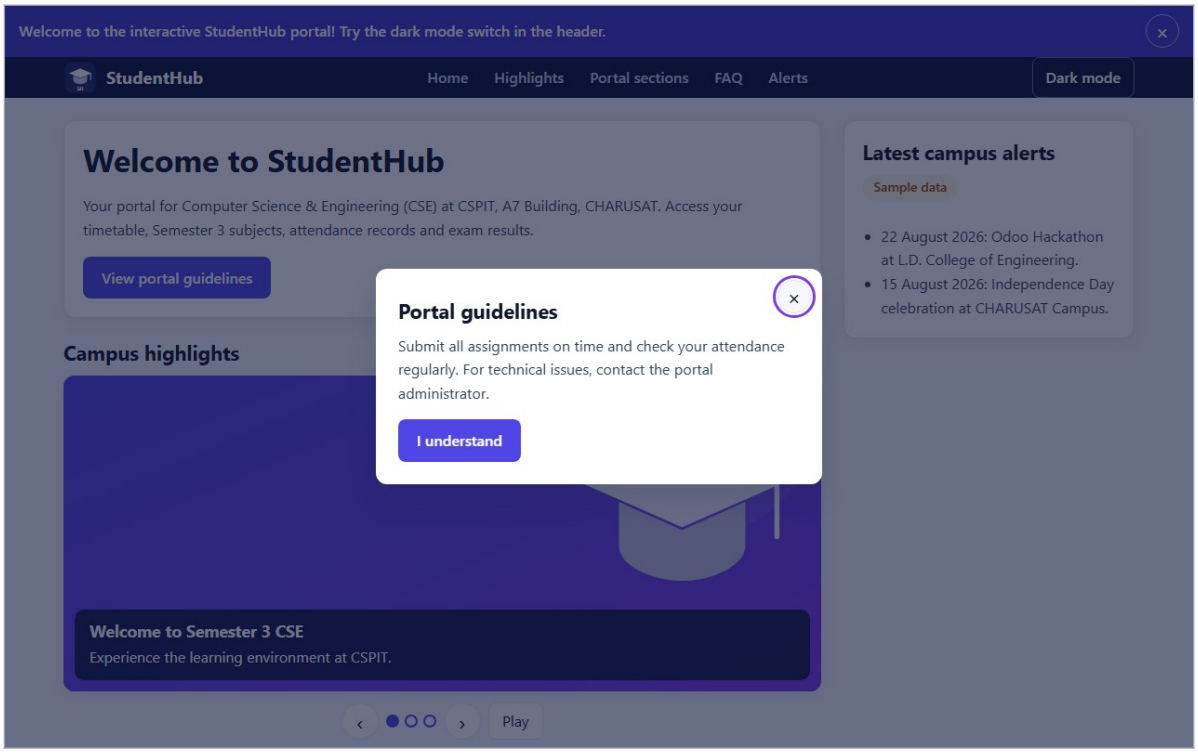


Figure 3: Portal guidelines modal open over the dimmed page; keyboard focus is on the close button (violet outline) – modal.png

7.4 Image/content slider

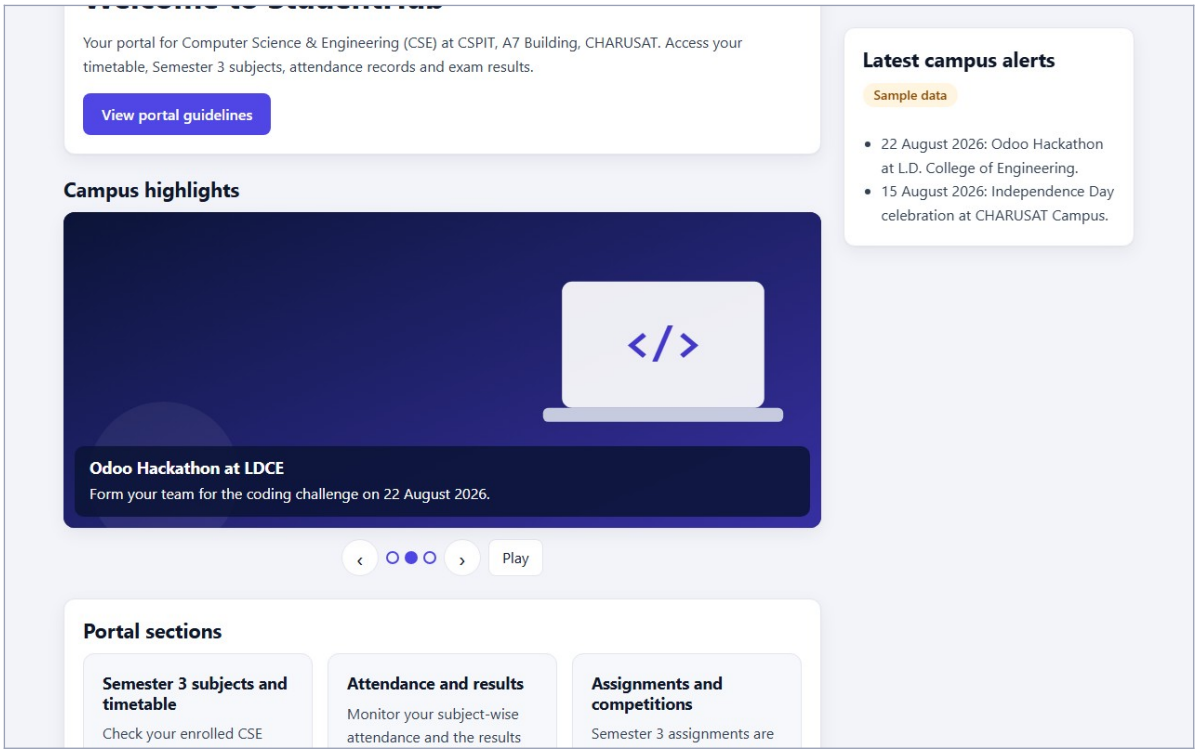


Figure 4: Slider after clicking Next – slide 2 of 3 is shown, the second dot is filled and auto-play has stopped (button shows Play) – slider.png

7.5 Notification banner dismissed and remembered

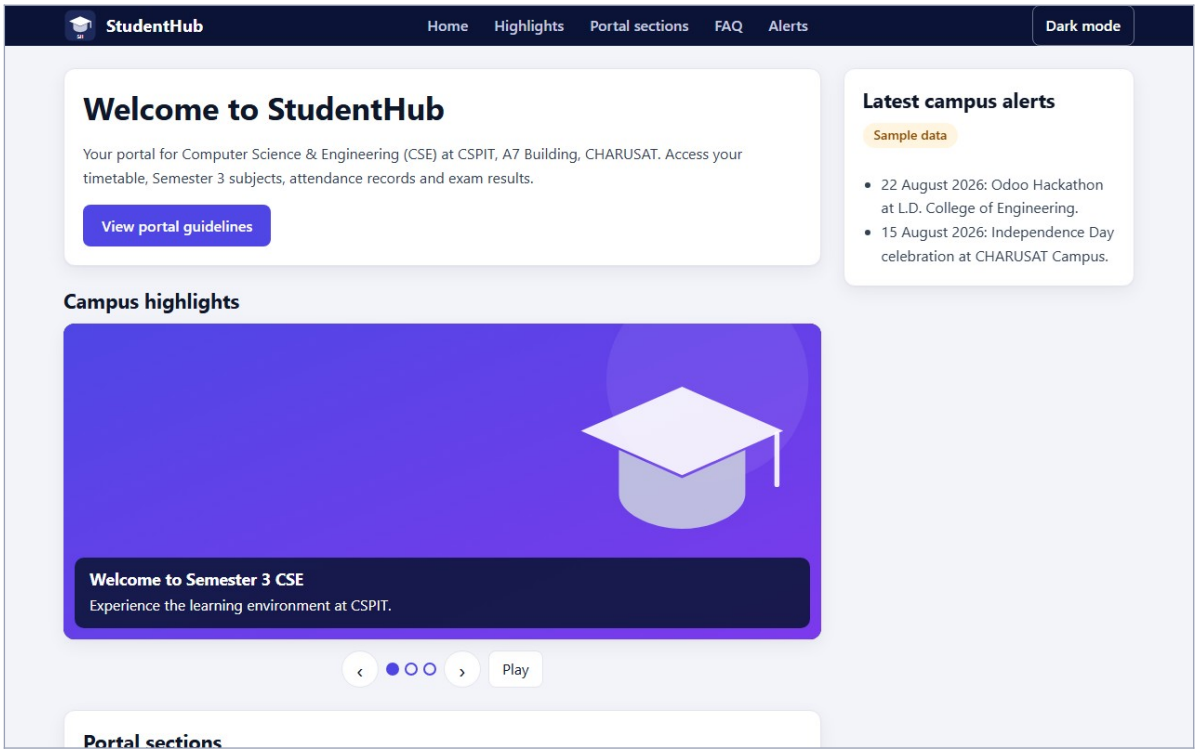


Figure 5: Page loaded again in a new Edge session after the banner was dismissed – the banner stays hidden – banner-dismissed-after-reload.png

7.6 Dark theme restored after reload

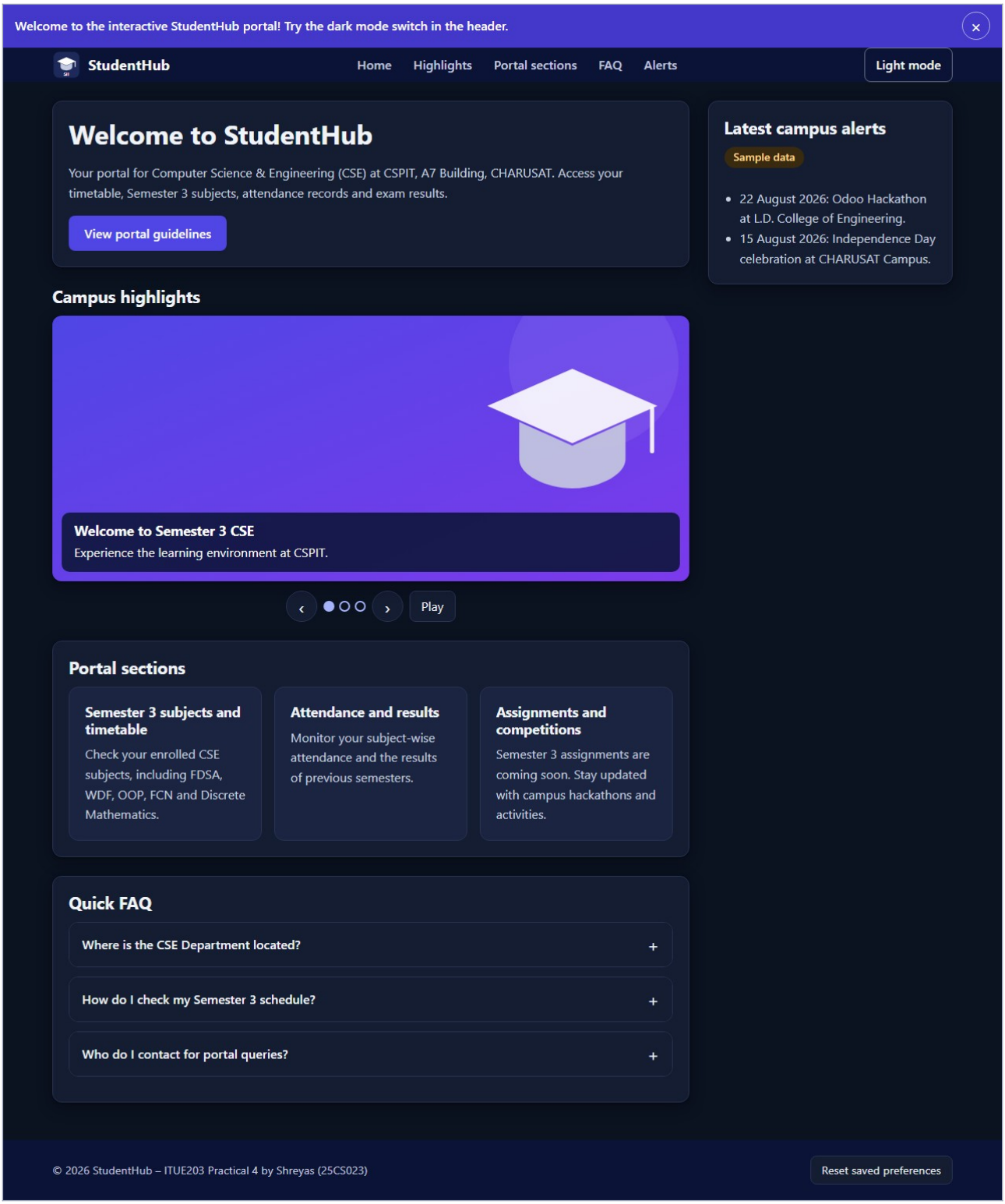
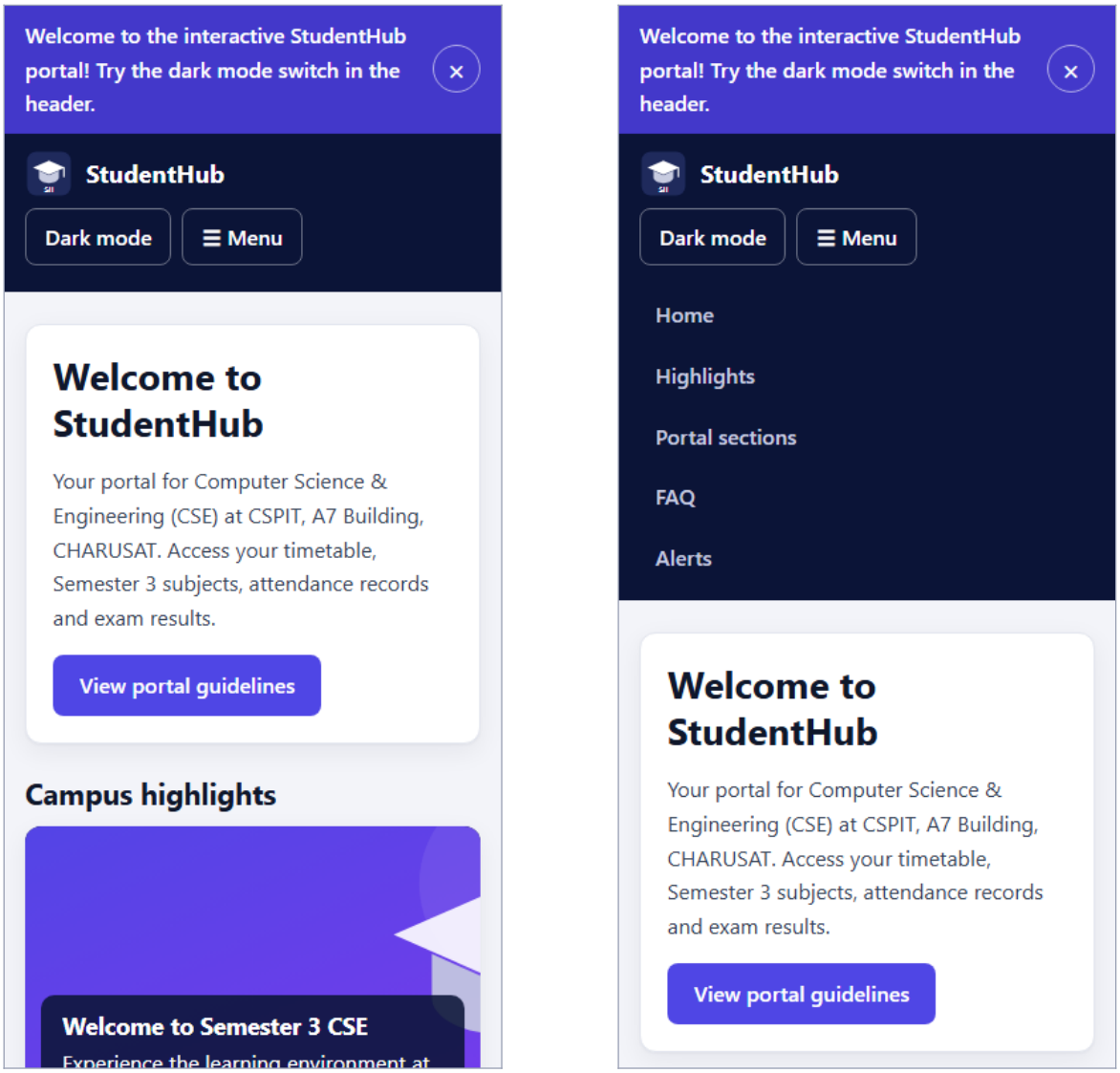


Figure 6: Complete page loaded again in a new Edge session after switching to the dark theme – the dark theme is restored and the button shows Light mode – dark-theme-after-reload.png

7.7 Hamburger menu at 375px



(a) Menu closed – top of page-mobile.png *(b) After clicking ☰ Menu – menu.png*

Figure 7: Hamburger menu at a width of 375px – the links are hidden until the Menu button is clicked

8. Source Code Excerpts

The excerpts are copied unchanged from the files in Practical-4/. Line numbers match the files; only the common leading indentation of each excerpt is removed.

8.1 index.html – banner, theme button and hamburger menu

```
15  <!-- 4. Dismissible notification banner -->
16  <div class="notification-banner" id="notification-banner" role="region" aria-label="Announcement">
17      <p>Welcome to the interactive StudentHub portal! Try the dark mode switch in the header.</p>
18      <button type="button" class="close-banner" id="close-banner" aria-label="Dismiss
    announcement">&times;</button>
19  </div>
20
21  <header class="site-header">
22      <div class="container site-header__inner">
23          <a class="brand" href="index.html"><img src="assets/images/studenthub-logo.svg" alt=""
    width="36" height="36"> StudentHub</a>
24          <div class="header-actions">
25              <!-- 6. Light/dark theme switcher -->
26              <button type="button" class="theme-toggle" id="theme-toggle" aria-pressed="false">Dark
    mode</button>
27              <!-- 5. Hamburger menu (shown below 768px) -->
28              <button type="button" class="hamburger-menu" id="hamburger-menu" aria-expanded="false"
    aria-controls="nav-links">&#9776; Menu</button>
29          </div>
30          <nav class="site-nav" aria-label="Main navigation">
31              <ul id="nav-links">
```

8.2 index.html – FAQ item and modal dialog

```
106  <div class="faq-item">
107      <h3><button type="button" class="faq-question" aria-expanded="false" aria-controls="faq-
     answer-1">Where is the CSE Department located?</button></h3>
108      <div class="faq-answer" id="faq-answer-1" hidden>The Department of Computer Science &amp;
     Engineering is in the A7 Building, CSPIT, CHARUSAT Campus.</div>
109  </div>
```

```
138  <!-- 2. Modal dialog (hidden until opened) -->
139  <div class="modal-overlay" id="custom-modal" hidden>
140      <div class="modal-content" role="dialog" aria-modal="true" aria-labelledby="modal-title">
141          <button type="button" class="close-modal" id="close-modal-btn" aria-label="Close
     guidelines">&times;</button>
142          <h2 id="modal-title">Portal guidelines</h2>
143          <p>Submit all assignments on time and check your attendance regularly. For technical issues,
     contact the portal administrator.</p>
144          <button type="button" class="btn-action" id="modal-ok-btn">I understand</button>
145      </div>
146  </div>
```

8.3 theme.js – localStorage helpers and theme restore

```
// Loaded in <head>: applies the saved theme before the page is drawn, so there is no flash of the wrong theme.
const THEME_KEY = 'sh_theme';

function readPref(key) {
    try { return localStorage.getItem(key); } catch (e) { return null; }   // storage may be blocked
}

function writePref(key, value) {
    try { localStorage.setItem(key, value); } catch (e) { /* preference simply is not remembered */ }
}

document.documentElement.setAttribute('data-theme', readPref(THEME_KEY) === 'dark' ? 'dark' : 'light');

window.toggleTheme = () => {
    const next = document.documentElement.getAttribute('data-theme') === 'dark' ? 'light' : 'dark';
    document.documentElement.setAttribute('data-theme', next);
    writePref(THEME_KEY, next);
    return next;
};
```

8.4 script.js – banner (localStorage) and theme button

```
document.addEventListener('DOMContentLoaded', () => {

    // 1. Notification banner: dismissible, and the dismissal is remembered in localStorage
    const BANNER_KEY = 'sh_banner_dismissed';
    const banner = document.getElementById('notification-banner');
    const closeBannerBtn = document.getElementById('close-banner');
    if (banner && closeBannerBtn) {
        if (readPref(BANNER_KEY) === '1') banner.hidden = true;
        closeBannerBtn.addEventListener('click', () => {
            banner.hidden = true;
            writePref(BANNER_KEY, '1');
            document.getElementById('main-content').focus({ preventScroll: true });   // focus does not get lost
        });
    }

    // 2. Theme toggle button: switches and saves the theme; label and aria-pressed follow the state
    const themeToggleBtn = document.getElementById('theme-toggle');
    const showThemeState = () => {
        const dark = document.documentElement.getAttribute('data-theme') === 'dark';
        themeToggleBtn.textContent = dark ? 'Light mode' : 'Dark mode';
        themeToggleBtn.setAttribute('aria-pressed', String(dark));
    };
    if (themeToggleBtn) {
        showThemeState();
        themeToggleBtn.addEventListener('click', () => {
            window.toggleTheme();
            showThemeState();
        });
    }
```

8.5 script.js – hamburger menu

```
// 3. Hamburger menu (below 768px): toggles the link list; Escape closes it
const hamburgerMenu = document.getElementById('hamburger-menu');
const navLinks = document.getElementById('nav-links');
const setMenu = (open) => {
    navLinks.classList.toggle('is-open', open);
    hamburgerMenu.setAttribute('aria-expanded', String(open));
};
if (hamburgerMenu && navLinks) {
    hamburgerMenu.addEventListener('click', () => setMenu(!navLinks.classList.contains('is-open')));
    navLinks.addEventListener('click', (e) => {
        if (e.target.closest('a')) setMenu(false);                          // close after choosing a link
    });
    document.addEventListener('keydown', (e) => {
        if (e.key === 'Escape' && navLinks.classList.contains('is-open')) {
            setMenu(false);
            hamburgerMenu.focus();
        }
    });
}
```

8.6 script.js – modal popup

```
const openModal = () => {
    lastFocused = document.activeElement;
    modal.hidden = false;
    closeModalBtn.focus();
};
const closeModal = () => {
    modal.hidden = true;
    if (lastFocused) lastFocused.focus();
};

if (modal && openModalBtn && closeModalBtn && modalOkBtn) {
    openModalBtn.addEventListener('click', openModal);
    closeModalBtn.addEventListener('click', closeModal);
    modalOkBtn.addEventListener('click', closeModal);
    modal.addEventListener('click', (e) => {
        if (e.target === modal) closeModal();                               // click on the dark overlay
    });
    document.addEventListener('keydown', (e) => {
        if (modal.hidden) return;
        if (e.key === 'Escape') {
            closeModal();
        } else if (e.key === 'Tab') {                                       // keep focus inside the dialog
            const focusable = [...modal.querySelectorAll('button')];
            const first = focusable[0];
            const last = focusable[focusable.length - 1];
            if (e.shiftKey && document.activeElement === first) {
                e.preventDefault();
                last.focus();
            } else if (!e.shiftKey && document.activeElement === last) {
                e.preventDefault();
                first.focus();
            }
        }
    });
```

8.7 script.js – collapsible FAQ

```
// 5. FAQ: one answer open at a time; aria-expanded and the hidden attribute follow the state
const faqQuestions = document.querySelectorAll('.faq-question');
faqQuestions.forEach((question) => {
    question.addEventListener('click', () => {
        const answer = document.getElementById(question.getAttribute('aria-controls'));
        const isOpen = question.getAttribute('aria-expanded') === 'true';

        // Close all others
        faqQuestions.forEach((q) => {
            q.setAttribute('aria-expanded', 'false');
            document.getElementById(q.getAttribute('aria-controls')).hidden = true;
        });

        if (!isOpen) {
            question.setAttribute('aria-expanded', 'true');
            answer.hidden = false;
        }
    });
});
```

8.8 script.js – slider

```
const dots = [...slides].map((slide, i) => {
    const dot = document.createElement('button');
    dot.type = 'button';
    dot.className = 'slider-dot';
    dot.setAttribute('aria-label', `Show slide ${i + 1}`);
    dot.addEventListener('click', () => {
        stopAutoplay();
        showSlide(i);
    });
    dotsBox.appendChild(dot);
    return dot;
});

function showSlide(index) {
    currentSlide = (index + slides.length) % slides.length;
    slides.forEach((slide, i) => {
        slide.style.transform = `translateX(${100 * (i - currentSlide)}%)`;
        slide.setAttribute('aria-hidden', String(i !== currentSlide));
    });
    dots.forEach((dot, i) => dot.setAttribute('aria-current', String(i === currentSlide)));
}

function startAutoplay() {
    timer = setInterval(() => showSlide(currentSlide + 1), 5000);
    pauseBtn.textContent = 'Pause';
    viewport.setAttribute('aria-live', 'off');                          // do not announce automatic
changes
}

function stopAutoplay() {
    clearInterval(timer);
    timer = null;
    pauseBtn.textContent = 'Play';
    viewport.setAttribute('aria-live', 'polite');                       // announce slides changed
by the user
}
```

```
pauseBtn.addEventListener('click', () => (timer ? stopAutoplay() : startAutoplay()));
if (reduceMotion) stopAutoplay(); else startAutoplay();
```

reduceMotion is set on line 123 with window.matchMedia('(prefers-reduced-motion: reduce)').matches.

8.9 style.css – dark theme variables, menu, FAQ icon, transitions and reduced motion

```
32  [data-theme="dark"] {
33      --bg: #0b1220;
34      --surface: #151d33;
35      --surface-2: #1b2440;
36      --border: #2a3555;
37      --text-dark: #f1f5f9;
38      --text: #cbd5e1;
39      --text-muted: #a3adc2;
40      --link: #a5b4fc;
41      --soft: #232c4d;
42      --dot: #a5b4fc;
43      --warning-soft: #3a2a10;
44      --warning-ink: #fcd38d;
45      --shadow: 0 1px 2px rgba(0, 0, 0, 0.3), 0 6px 20px rgba(0, 0, 0, 0.3);
46      --shadow-hover: 0 18px 30px rgba(0, 0, 0, 0.45);
47  }
```

```
134  .site-nav { width: 100%; }
135  .site-nav ul { display: none; flex-direction: column; gap: 4px; margin: 0; padding: 8px 0 0; list-
     style: none; }
136  .site-nav ul.is-open { display: flex; }
```

```
148  .articles-flex { display: flex; flex-wrap: wrap; gap: 16px; }
149  .articles-flex article {
150      flex: 1 1 100%; padding: 20px;
151      border: 1px solid var(--border); border-radius: var(--radius); background: var(--surface-2);
152      transition: transform 0.3s ease, box-shadow 0.3s ease, border-color 0.3s ease;
153  }
154  .articles-flex article:hover { transform: translateY(-6px); box-shadow: var(--shadow-hover); border-
     color: var(--indigo); }
```

```
197  .faq-question::after { content: "+"; font-size: 1.4rem; font-weight: 400; transition: transform 0.3s
     ease; }
198  .faq-question[aria-expanded="true"]::after { transform: rotate(45deg); }
```

```
243  /* Less motion when the system asks for it: no sliding or lifting, colour transitions stay.
244     (script.js also starts the slider paused in this case.) */
245  @media (prefers-reduced-motion: reduce) {
246      .slide { transition: none; }
247      .modal-overlay { animation: none; }
248      .articles-flex article:hover,
249      .btn-action:hover,
250      .slider-btn:hover { transform: none; }
251  }
```

9. Testing Results, Accessibility Checks and Limitations

9.1 Static checks – source/check_page.py

A local script written for this practical (Python standard library only) reads index.html, style.css, theme.js and script.js. It is not the W3C validator. Command and result:

```
python Practical-4/source/check_page.py
15 of 15 static checks passed
```

#	Check	Result
1–3	No inline style attributes, no onclick or other inline event handlers, no <style> elements	Pass
4	No external URLs (fonts, images and scripts are local)	Pass
5	All local links, images, CSS and JS files exist, including #fragment targets	Pass
6–7	No duplicate ids; exactly one h1	Pass
8–9	Every id used by script.js exists; every aria-controls points to an existing id	Pass
10	Skip link targets main#main-content	Pass
11–13	Every image has alt; every button has visible text or an aria-label; all buttons have type="button"	Pass
14	Theme saved and restored with localStorage	Pass
15	JavaScript uses ES6+ (const/let, arrow functions, template literals; no var)	Pass

Control test: the same rules run on the original Practical 4 page flagged 3 inline styles, 1 onclick attribute, 2 <style> elements and 4 external URLs, so the script detects these problems.

9.2 Colour contrast – source/contrast_check.py

WCAG 2.1 contrast ratios were calculated for 22 colour pairs taken from style.css (minimum 4.5:1 for text, 3:1 for the slider dots and the focus outline). All 22 pairs pass. Selected results:

Colour pair	Theme	Ratio	Minimum
Body text on cards (#334155 on #ffffff)	Light	10.35:1	4.5:1
Links on cards (#4338ca on #ffffff)	Light	7.90:1	4.5:1
Banner text (white on #4338ca)	Both	7.90:1	4.5:1
Primary buttons (white on #4f46e5)	Both	6.29:1	4.5:1
Slide caption text, worst case	Both	12.92:1	4.5:1
Body text on cards (#cbd5e1 on #151d33)	Dark	11.27:1	4.5:1
Sample-data badge (#fcd38d on #3a2a10)	Dark	9.77:1	4.5:1
Slider dots (#a5b4fc on #0b1220)	Dark	9.39:1	3:1
Focus outline (#7c3aed on #0b1220)	Dark	3.29:1	3:1

9.3 Component tests in Microsoft Edge

Method: a temporary Python test script (kept outside the submitted files) opened copies of index.html in Microsoft Edge 156 in headless mode. A script added to each copy called element.click() and dispatched keydown events (Escape, Tab, Shift+Tab), which run the page's own event listeners, and then checked the DOM state, the focused element and localStorage. An error recorder captured page errors, failed file loads and console.error messages. For the persistence tests the page was opened again in a new Edge session with the same browser profile. Result: 56 of 56 checks passed.

Test case	What was checked	Result
Page load	Page visible, stylesheet applied, banner shown, 3 slides and 3 dots, desktop menu, all slide images loaded	7 / 7
FAQ	Answers hidden at start; question 1 opens; question 2 opens and question 1 closes; clicking an open question closes it	5 / 5
Modal	Opens with focus on ×; Tab and Shift+Tab wrap inside; Escape closes and focus returns; overlay click and “I understand” close it	8 / 8
Slider (normal motion)	Auto-play running; moves to slide 2 after 5 s; Next/Previous/dots and wrap-around; Next stops auto-play; Play restarts it; aria-hidden, aria-current and aria-live states	10 / 10
Slider (reduced motion)	This PC reports reduced motion; slider starts paused; no automatic change; Next works; colour transition still active	6 / 6
Hamburger menu (375px)	Links hidden and Menu shown; opens with aria-expanded=true; Escape closes and focuses Menu; choosing a link closes it; no horizontal overflow	6 / 6
Banner	Dismissed; saved in localStorage; focus moved to main content	4 / 4
Banner after reload	Still hidden in a new Edge session	2 / 2
Theme	Light by default; switches to dark; aria-pressed=true and label Light mode; saved in localStorage	5 / 5
Theme after reload	Dark theme and button state restored in a new Edge session	3 / 3

Every test case also checked that no page or console errors occurred (included in the counts above). Total: 56 of 56.

9.4 Problems found and fixed during testing and screenshot review

- **Dark-theme slider dots:** measured 2.98:1 against the dark background, below the 3:1 minimum. A separate --dot colour variable (#a5b4fc in the dark theme) raised it to 9.39:1.
- **Reduced motion:** the first version switched off every transition when the system asked for reduced motion, so the theme and button colour transitions disappeared. The rule now switches off only movement (slide, lift and fade); the slider starts paused instead.
- **Spacing:** the welcome card touched the header because a padding rule was overridden; the layout now uses margins.

9.5 Accessibility summary

- Every interactive control is a native button or link with a visible label or an aria-label, so it can be reached with Tab.
- ARIA states follow the components: aria-expanded (FAQ, menu), aria-pressed (theme), aria-current (slider dots), aria-hidden (inactive slides), aria-modal and aria-labelledby (dialog).
- Escape closes the menu and the modal; focus moves into the modal, stays inside it, and returns to the opening button.
- The slider can be paused, and it starts paused when the system asks for reduced motion.
- All 22 checked colour pairs meet the WCAG 2.1 AA contrast minimum in both themes; a visible violet focus outline is shown on keyboard focus.

9.6 Limitations – not tested

- **Physical input:** the tests sent clicks and key presses through JavaScript; nobody operated the page with a real mouse and keyboard during the recorded tests.
- **Enter and Space:** native activation of buttons with Enter or Space was not simulated; the components use real button elements, which browsers activate from the keyboard.
- **Screen readers:** not tested.
- **Other browsers and devices:** only Microsoft Edge on Windows 11 was used; the 375px tests used a narrow frame, not a real phone.
- **W3C validation:** the page was not submitted to the W3C Markup Validation Service.

10. Results and Conclusion

The StudentHub home page in Practical 4 now has six working interactive components: a collapsible FAQ, a modal popup, an image/content slider, a dismissible notification banner, a responsive hamburger menu and a light/dark theme switcher. They are written in ES6+ JavaScript in separate files, select and change the page through the DOM API, and react to clicks and key presses through addEventListener; the page has no inline styles or event handlers and works offline.

Both extensions from the practical list are included: transition effects on cards and buttons (intermediate), and theme and banner preferences stored in localStorage and restored on page load (advanced). The static checks (15 of 15), the colour-contrast check (22 of 22) and the scripted browser tests in Microsoft Edge (56 of 56, with no console errors) all passed, and the screenshots show each component working.

Remaining limitations: testing with a real mouse and keyboard by a person, screen readers, other browsers, real phones and official W3C validation were not carried out. Through this practical, JavaScript was applied to improve the user experience and client-side interactivity of StudentHub while keeping it accessible.

11. References

- ITUE203 Web Development Frameworks – Practical List 2026-27, Practical 4: “JavaScript DOM Manipulation, Event Handling, and UI Interactivity”, CSPIT, CHARUSAT.
- MDN Web Docs – Document Object Model (DOM): https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- MDN Web Docs – EventTarget.addEventListener(): https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
- MDN Web Docs – Window.localStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- MDN Web Docs – prefers-reduced-motion: https://developer.mozilla.org/en-US/docs/Web/CSS/@media/prefers-reduced-motion
- W3C WAI-ARIA Authoring Practices Guide – Accordion, Dialog (Modal) and Carousel patterns: https://www.w3.org/WAI/ARIA/apg/patterns/
- W3C, Web Content Accessibility Guidelines (WCAG) 2.1: https://www.w3.org/TR/WCAG21/
- StudentHub repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (this practical’s changes are uploaded separately).
- Practical-4/README.md; Practical-4/source/check_page.py and contrast_check.py.